

RAPPORT DE TP

Cloud & Virtualisation

VMware ESXi vs Proxmox VE

Module : Cloud & Infrastructure
1ère année Cybersécurité — 2025/2026

1. Introduction

Ce TP fait suite à l'installation de VMware ESXi et Proxmox VE réalisée lors des séances précédentes, en virtualisation imbriquée (nested) sous VMware Workstation. Le but était de prendre en main les deux interfaces d'administration, de créer des machines virtuelles, de configurer le réseau et le stockage, de manipuler les snapshots, de consulter les logs, de gérer les comptes/permissions, puis de réaliser un mini-laboratoire de cybersécurité.

2. Partie A — VMware ESXi

Connexion à l'interface : nous avons relevé l'adresse IP de l'hôte ESXi sur l'écran de la console (DCUI), puis nous y avons accédé via `https://<IP_ESXi>/` (par exemple 192.168.1.100). Le certificat auto-signé a généré un avertissement, accepté pour continuer. Connexion avec le compte root.

Dashboard : le tableau de bord affiche la version d'ESXi, l'usage CPU et mémoire, la capacité du datastore et le nombre de VMs. Nous avons parcouru les sections Host, Virtual Machines, Storage, Networking, Manage et Monitor.

Réseau : dans Networking, nous avons retrouvé le vSwitch0 avec son port group par défaut (VM Network) et le Management Network. Nous avons créé un nouveau port group nommé Lab-Isolated sur le vSwitch0, qui servira pour le mini-lab. Les politiques de sécurité par défaut (promiscuous mode, MAC changes, forged transmits réglés sur Reject) ont été observées : elles empêchent une VM d'écouter le trafic des autres ou de falsifier son adresse MAC.

Création d'une VM : une ISO Debian a été téléversée dans le datastore (Storage → datastore1 → Datastore browser), puis une VM Test-Debian-ESXi a été créée avec 1 vCPU, 1 Go de RAM, 8 Go de disque en thin provisioning, connectée au réseau VM Network. Après démarrage (Power On), la console affiche bien l'écran de boot/installation de l'ISO.

Snapshots : un snapshot Avant-installation a été pris (Actions → Snapshots → Take snapshot), puis vérifié dans Manage snapshots. À noter que sur un ESXi standalone (sans vCenter), il n'y a pas de fonction de clonage native dans l'interface web.

Monitoring et logs : dans Monitor → Performance, on retrouve les graphiques temps réel (CPU, RAM, réseau, disque). Dans Monitor → Events, l'historique des actions (démarrages, connexions, erreurs) est consultable, et constitue un outil d'audit important. Les principaux fichiers de logs sont vmkernel.log (matériel), hostd.log (actions d'admin), auth.log (authentification) et shell.log (commandes SSH).

Comptes et durcissement : un utilisateur auditeur a été créé avec le rôle Read-Only (Manage → Security & Users → Users), afin d'illustrer le principe du moindre privilège. Côté services (Manage → Services), SSH et l'ESXi Shell doivent rester désactivés sauf besoin ponctuel, le DCUI doit rester actif, et le NTP doit être activé pour garantir la cohérence des horodatages dans les logs.

CLI : avec SSH activé temporairement, nous avons testé des commandes comme `esxcli system version get`, `esxcli network vswitch standard list`, `esxcli storage filesystem list` ou `vim-cmd vmsvc/getallvms`. SSH a été désactivé dès la fin de la manipulation.

3. Partie B — Proxmox VE

Connexion : l'adresse IP de la VM Proxmox a été relevée via `ip a`, puis l'interface a été ouverte sur `https://<IP_Proxmox>:8006/` (par exemple 192.168.1.200), avec le compte root et le realm PAM (Linux).

Arborescence : l'interface est organisée en Datacenter → Node → VM/CT. Le résumé du nœud affiche l'usage CPU/RAM, l'espace disque, l'uptime, ainsi que les versions de Proxmox VE, du noyau et de QEMU.

Réseau : dans Node → Network, on retrouve le bridge principal `vmbr0` relié à l'interface physique. Nous avons créé un second bridge `vmbr1` (Create → Linux Bridge) avec l'adresse 10.10.10.1/24 et sans port physique (bridge ports vide), ce qui crée un réseau totalement isolé d'Internet — idéal pour le mini-lab de la partie 4.

Stockage : les pools par défaut sont local (répertoire, pour les ISO et sauvegardes) et local-lvm (LVM-thin, pour les disques de VM). Une ISO Debian a été téléversée sur local.

Création d'une VM : la VM Test-Debian-PVE (ID 100) a été créée avec le bouton Create VM : machine q35, BIOS OVMF (UEFI) ou SeaBIOS, disque VirtIO de 8 Go sur local-lvm, 1 vCPU de type host (important pour le nested), 1024 Mo de RAM, et carte réseau VirtIO sur `vmbr0`. Après démarrage, la console noVNC affiche le boot sur l'ISO.

Conteneur LXC (optionnel) : un conteneur CT-test (ID 200) a été créé à partir du template `debian-12-standard`, avec 512 Mo de RAM, 4 Go de disque et le bridge `vmbr0`. Par défaut, Proxmox crée des conteneurs non privilégiés : le root du conteneur est mappé sur un utilisateur non-root de l'hôte, ce qui limite fortement les risques d'évasion.

Snapshots et sauvegardes : un snapshot état-initial a été pris, et une sauvegarde manuelle (Backup → Backup now) a été réalisée sur le storage local, en mode Snapshot avec compression ZSTD. La différence essentielle : un snapshot reste lié au disque de la VM (perdu en cas de crash), alors qu'une sauvegarde `vzdump` est un fichier autonome stocké séparément, donc réellement exploitable en cas de sinistre.

Monitoring et journaux : l'onglet Summary de la VM affiche les graphiques temps réel ; Node → System → Syslog donne accès aux logs système Linux ; le Task Log liste l'historique des opérations effectuées.

RBAC : un utilisateur `etudiant1` a été créé (Datacenter → Permissions → Users → Add, realm pve), puis une permission lui a été attribuée sur le chemin `/vms/100` avec le rôle `PVEVMUser` (utiliser la VM sans pouvoir la configurer). Le RBAC de Proxmox est beaucoup plus granulaire que la gestion de comptes basique d'un ESXi standalone : rôles personnalisables, granularité par objet, plusieurs realms d'authentification.

CLI : commandes testées — `pveversion`, `qm list` / `qm status 100` / `qm start 100`, `qm snapshot 100 snap1`, `pct list`, `pvesm status`, `vzdump 100 --storage local --mode snapshot --compress zstd`, et `pvesh get /nodes` pour l'API REST.

4. Partie D — Mini-lab cybersécurité (réalisé sur Proxmox)

Le mini-lab a été réalisé sur Proxmox VE, en utilisant le bridge isolé `vmbr1` (10.10.10.0/24, sans accès Internet).

4.1 Mise en place

- VM-Server : Debian 12, 1 vCPU, 1 Go RAM, 8 Go disque, connectée à vmbr1, IP statique 10.10.10.10/24.
- VM-Client : mêmes caractéristiques, connectée à vmbr1, IP statique 10.10.10.20/24.

La configuration IP statique a été faite en éditant /etc/network/interfaces sur chaque VM, suivi d'un systemctl restart networking.

4.2 Test de connectivité et isolation

- Depuis VM-Client : ping -c 4 10.10.10.10 → succès, 0 % de perte (les deux VMs communiquent bien).
- Depuis VM-Client : ping vers 8.8.8.8 → échec, ce qui confirme que vmbr1 est bien isolé d'Internet.

4.3 Trois mesures de sécurité appliquées sur VM-Server

1. Création d'un utilisateur non-root operateur, ajouté au groupe sudo (adduser operateur ; usermod -aG sudo operateur).
2. Installation et activation du pare-feu UFW : blocage de tout le trafic entrant par défaut, autorisation du SSH uniquement depuis le réseau 10.10.10.0/24 (ufw allow from 10.10.10.0/24 to any port 22), puis ufw enable.
3. Désactivation de la connexion root par SSH : modification de PermitRootLogin à no dans /etc/ssh/sshd_config, puis systemctl restart sshd.

Vérification depuis VM-Client : ssh operateur@10.10.10.10 fonctionne, ssh root@10.10.10.10 est refusé, et le ping continue de fonctionner.

4.4 Snapshot, incident simulé, restauration

4. Prise d'un snapshot etat-securise de VM-Server, représentant l'état sécurisé après les 3 mesures.
5. Simulation d'un incident : désactivation du pare-feu (ufw disable) et réactivation du login root SSH (PermitRootLogin yes + redémarrage de sshd).
6. Restauration du snapshot etat-securise depuis l'interface Proxmox.

Résultat après restauration : sudo ufw status indique le pare-feu actif, et grep PermitRootLogin /etc/ssh/sshd_config renvoie no. La VM est bien revenue à son état sécurisé d'origine.

4.5 Schéma du mini-lab

Architecture : un bridge isolé vmbr1 (10.10.10.0/24, sans uplink) hébergé sur Proxmox, avec deux VMs connectées : VM-Server (10.10.10.10) sécurisée (utilisateur operateur + sudo, UFW actif limitant le SSH au LAN, root SSH désactivé) et VM-Client (10.10.10.20) utilisée pour les tests. Aucune des deux VMs n'a d'accès à Internet. Le snapshot etat-securise sert de point de restauration de référence.

5. Comparaison ESXi vs Proxmox VE

Critère	ESXi	Proxmox VE
Type	Type 1 (VMkernel)	Type 1 (Debian + KVM)

Licence	Gratuit limité / payant (vSphere)	Open source gratuit (AGPLv3)
Conteneurs	Non	Oui (LXC)
Sauvegarde intégrée	Non (Veeam, ghettoVCB...)	Oui (vzdump)
RBAC	Basique sans vCenter	Complet et natif
Clustering	Payant (vCenter)	Gratuit et natif
Surface d'attaque admin	Réduite (interface minimale)	Plus large (Debian complet)
Réseau	vSwitch, port groups	Bridges Linux, VLAN, Open vSwitch
Stockage	VMFS, NFS, vSAN	LVM-thin, ZFS, Ceph, NFS...
Mises à jour	VMware Update Manager	apt update / dist-upgrade

Pour un labo académique ou de pentest, Proxmox est plus adapté (gratuit, complet, conteneurs, réseau flexible). Pour un environnement SOC ou une grande entreprise déjà sur VMware, ESXi/vSphere reste pertinent grâce à sa surface d'attaque réduite, mais nécessite vCenter (payant) pour un RBAC et un clustering complets.

6. Réponses aux questions

6.1 Questions techniques

7. Type 1 vs Type 2 : un hyperviseur Type 1 (ex. ESXi) s'installe directement sur le matériel sans OS intermédiaire, ce qui donne de meilleures performances et une meilleure isolation. Un Type 2 (ex. VMware Workstation) tourne comme une application au-dessus d'un OS existant — plus simple à installer, adapté au test.
8. vSwitch vs bridge Linux : les deux jouent le rôle d'un commutateur virtuel reliant les VMs au réseau physique. Le vSwitch est propre à VMkernel et s'organise en port groups ; le bridge Linux (vbr0) utilise les outils réseau natifs de Linux et se configure dans `/etc/network/interfaces`. Fonctionnellement, ils sont équivalents.
9. Thin vs thick provisioning : en thin, le disque ne consomme que l'espace réellement utilisé et grossit à la demande (économie d'espace, légère perte de perf). En thick, tout l'espace est alloué dès la création (meilleure performance, plus d'espace utilisé immédiatement).
10. Snapshot vs sauvegarde : un snapshot capture l'état d'une VM à un instant donné mais reste dépendant du disque original — si ce disque est perdu, le snapshot l'est aussi. Une sauvegarde est un fichier autonome stocké à part, exploitable même en cas de perte totale de la VM.
11. Lister les VMs : sur ESXi, `esxcli vm process list` ou `vim-cmd vmsvc/getallvms` ; sur Proxmox, `qm list` (VMs) et `pct list` (conteneurs).

6.2 Questions de cybersécurité

5 mesures de durcissement pour un ESXi en production :

- Mot de passe root complexe et comptes nominatifs avec rôles limités.
- Désactiver SSH et l'ESXi Shell sauf besoin ponctuel, avec timeout du shell.

- Activer le lockdown mode (avec vCenter).
- Configurer un serveur NTP pour des logs cohérents dans le temps.
- Envoyer les logs vers un syslog/SIEM centralisé.

Moindre privilège et RBAC Proxmox : le moindre privilège consiste à ne donner à chaque utilisateur que les droits nécessaires à sa tâche. Le RBAC de Proxmox permet d'attribuer des rôles précis (PVEVMUser, PVEAuditor...) sur des chemins précis (ex. /vms/100), donc un utilisateur peut être limité à une seule VM sans accès au reste de l'infrastructure.

Risques d'une interface d'admin exposée sur Internet : brute force sur les identifiants, exploitation de failles connues, interception du trafic. Protections : VPN/bastion, restriction par IP via firewall, authentification forte, mises à jour régulières, surveillance des logs de connexion.

LXC non privilégié pour un service critique : l'isolation par namespaces/cgroups réduit fortement le risque d'évasion, mais le noyau reste partagé avec l'hôte — une faille noyau pourrait permettre une évasion, ce qui est impossible avec une VM ayant son propre noyau. Pour un service vraiment critique, une VM complète reste préférable ; un conteneur convient à des services internes de criticité modérée.

Centralisation des logs ESXi vers un SIEM : se configure via Manage → System → Advanced Settings → Syslog.global.logHost. Intérêt : même si un attaquant compromet l'hôte et efface les logs locaux, les copies déjà envoyées au SIEM restent disponibles pour l'investigation et la corrélation avec d'autres systèmes.

6.3 Exercice comparatif — cas d'une PME

Pour une PME de 50 employés (3 serveurs à virtualiser, budget limité, 1 admin à mi-temps, besoin de sauvegardes automatisées), Proxmox VE est le choix le plus adapté. Il est gratuit même pour le clustering, contrairement à ESXi dont les fonctions avancées nécessitent une licence vSphere payante. Son interface complète (RBAC, firewall, stockage avancé) permet à un seul admin de tout gérer facilement. Il intègre nativement vzdump pour des sauvegardes planifiées et compressées, alors qu'ESXi ne propose rien d'équivalent sans outil tiers payant (Veeam). Côté sécurité, étant basé sur Debian, Proxmox bénéficie de mises à jour fréquentes via apt et d'un RBAC granulaire pour créer des comptes limités par serveur. ESXi/VMware garde un meilleur support éditeur, mais celui-ci est payant. Au global, Proxmox offre le meilleur rapport coût/fonctionnalités pour ce contexte.

6.4 Questions de réflexion

- Un Type 1 est préféré en production car il offre de meilleures performances (accès direct au matériel) et une meilleure isolation (pas d'OS hôte à compromettre).
- Accumuler des snapshots sans les consolider augmente l'espace disque consommé, dégrade les performances (chaque écriture traverse plusieurs niveaux de delta) et peut saturer le datastore.
- Séparer le réseau de management de celui des VMs limite la surface d'attaque : une VM compromise ne peut pas atteindre directement l'interface d'administration.
- Laisser le mode promiscuous activé permet à une VM de capturer le trafic des autres VMs du même segment (sniffing, fuite d'identifiants).
- Un SIEM peut corréler les logs (ex. tentatives SSH répétées, créations de comptes inhabituelles) pour détecter une compromission en cours.
- Créer des comptes nominatifs plutôt que partager root permet de tracer qui a fait quoi et de révoquer un accès individuellement.

- Séparer le stockage des sauvegardes de celui des VMs est essentiel : en cas de panne ou de ransomware sur le stockage principal, les sauvegardes séparées restent intactes.
- Pour un labo de forensic, Proxmox est préférable : réseaux isolés faciles à créer, snapshots rapides, flexibilité réseau, sans coût de licence.
- Proxmox étant un Debian complet, sa surface d'attaque est plus large (services Linux, paquets à maintenir) : il faut appliquer les bonnes pratiques de durcissement Linux classiques en plus de la config Proxmox.
- Le changement de politique de licences de Broadcom (ex-VMware), plus coûteux et sans offre gratuite équivalente, pousse les PME vers des alternatives open source comme Proxmox.
- Le snapshot ne protège pas contre une exfiltration de données déjà effectuée avant la restauration : il restaure l'état local mais ne récupère pas les données déjà copiées par l'attaquant.
- Trois mesures supplémentaires pour le mini-lab : fail2ban contre le brute force SSH, journalisation auditd envoyée vers un serveur externe au lab, et chiffrement du disque des VMs.
- Pour un rapport d'audit, on documenterait l'architecture avec un schéma de topologie (IPs, bridge, rôle de chaque VM), un tableau des mesures de sécurité appliquées, et une chronologie des tests (connectivité, isolation, snapshot, incident, restauration).

7. Conclusion

Ce TP nous a permis de passer de l'installation des hyperviseurs à leur réelle prise en main : création de VMs, configuration réseau et stockage, snapshots/sauvegardes, gestion des comptes et des permissions, et lecture des journaux. Le mini-lab cybersécurité a montré concrètement l'intérêt du moindre privilège, de la segmentation réseau, du durcissement (pare-feu, accès SSH limité) et du test effectif d'une restauration de snapshot. Entre ESXi et Proxmox, aucune solution n'est universellement meilleure : Proxmox se distingue par son coût nul et sa richesse fonctionnelle (idéal pour un labo ou une PME), tandis qu'ESXi conserve l'avantage d'une surface d'attaque réduite, mais au prix de vCenter pour exploiter pleinement le RBAC et le clustering.